Agentic Cloud Cluster

A Go Distributed Cluster Managing Framework with PPO-based Scheduling

A Project Report

Submitted by

Sarthak Siddhpura

Enrollment Number: AU2240041

in partial fulfillment for the award of the degree

BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering

Internal Faculty Mentor

Professor Sanjay Chaudhary

Ahmedabad
University

School of Engineering and Applied Science (SEAS)

Ahmedabad University

Ahmedabad, Gujarat

May, 2026

DECLARATION

I hereby declare that the project entitled "**Agentic Cloud Cluster: A Go Distributed Cluster Managing Framework with PPO-based Scheduling**" submitted for the B. Tech. degree is my original work and the project has not formed the basis for the award of any other degree, diploma, fellowship, or any other similar title.

Signature of Student

Date:

Place: Ahmedabad

CERTIFICATE

This is to certify that the project titled "**Agentic Cloud Cluster: A Go Distributed Cluster Managing Framework with PPO-based Scheduling**" is the bona fide work carried out by **Sarthak Siddhpura**, Enrollment Number **AU2240041**, a student of B. Tech. in Computer Science and Engineering at the School of Engineering and Applied Science, Ahmedabad University, during the academic year 2025–2026, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology.

This project was done under the supervision of the faculty mentor **Professor Sanjay Chaudhary**.

Signature of Faculty Mentor

Date:

Place: Ahmedabad

Abstract

Agentic Cloud Cluster is a Go-based distributed task execution and scheduling framework. The system operates a master node, multiple worker nodes, execution of tasks based on Docker, persistent task state, worker heartbeats, task retries, and a scheduler interface, capable of switching between baseline and learned policies. What is most valuable about this project is that the cluster is not only running tasks but the scheduling is a learning problem. The framework combines a Proximal Policy Optimization (PPO) scheduler that is trained on cluster traces and deployed on top of a safe Go scheduler boundary.

The report elaborates on the project as it is on the ground. It initially drives the need to schedule distributed clusters and then justifies why reinforcement learning is appropriate in situations where it is optimal to modify resource state based on factors like scheduling. The methodology follows a sequence of steps beginning with tabular Q-learning, and then to Deep Q-Networks and finally to PPO. The PPO policy monitors task requirements and features of worker resources, selects a worker, obtains a shaped reward, and improves using clipped policy-gradient updates. Its implementation is based on Go in the master-worker control plane, and Python/PyTorch in the PPO service.

The last system was tested by using offline trace replay and end-to-end cluster campaigns. The convergence point of the PPO training curve is through the reduction in the policy and value losses, whereas the benchmark plots compare PPO to the baseline schedulers in terms of resource and workload patterns. The findings indicate that the acquired scheduler is most effective when it is necessary to schedule pressure (such as burst and overload situations) and simple baselines can still be competitive in cases of low load. This renders the project a viable exploration of the areas in which reinforcement learning assists in systems work and where conservative fallback design is still required.

Acknowledgements

I would like to express my deepest gratitude to my internal faculty mentor, **Professor Sanjay Chaudhary**, for his invaluable guidance, constant encouragement, and insightful feedback throughout the development of this project. His expertise and support were instrumental in shaping the technical direction and ensuring the successful completion of this work.

I would also like to extend my sincere thanks to the School of Engineering and Applied Science (SEAS), Ahmedabad University, for providing the academic environment and resources necessary for this research.

I am profoundly grateful to my **family** for their unwavering support, patience, and belief in my endeavors. Their encouragement has been a constant source of strength throughout my academic journey.

Finally, I would like to thank my friends and colleagues for their critical suggestions, technical discussions, and refinement comments, which greatly helped in the overall ideation and improvement of this project.

Table of Contents

List of Figures

List of Tables

Gantt Chart

Table 0.0.1: Project timeline

Activity	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15	W16	W17
Problem study and literature review	X	X	X														
Go master-worker framework		X	X	X	X												
Worker execution, heartbeats, and persistence				X	X	X	X	X									
Baseline schedulers and scheduler interface						X	X	X	X								
PPO training pipeline and trace replay								X	X	X	X	X					
PPO integration with Go control plane											X	X	X				
Benchmark campaigns and result analysis													X	X	X	X	
Report writing and final polishing																	X

The timeline now spans 17 weeks. The early weeks established the distributed cluster baseline, the middle weeks focused on PPO training and integration, and week 17 was reserved exclusively for documentation and polishing.

Chapter 1

Introduction

Modern computing workloads are rarely executed on a single machine. A useful system must accept many tasks, understand what resources each task needs, select a suitable worker, run the task safely, and store the result. This becomes harder when workers are heterogeneous. One worker may have more CPU, another may have more memory, and the best choice can change every few seconds as tasks start and finish.

The project developed in this work is called **Agentic Cloud Cluster**. It is a Go-based distributed cluster managing framework that schedules Docker-based tasks across worker nodes. The focus of the project is the cluster framework and the PPO-based scheduler. Other surrounding utilities are treated only as supporting parts of the system.

1.1 | Project Definition

The project can be defined as follows:

> To build a distributed task execution framework in Go and improve task placement decisions using a reinforcement learning scheduler based on Proximal Policy Optimization.

The system has a master node and multiple worker nodes. The master accepts tasks, tracks workers, stores task state, and asks a scheduler to select a worker. The worker executes the task using Docker containers and reports status back to the master. The scheduling algorithm can be changed without changing the worker execution logic.

1.2 | Motivation

The first scheduling solution most systems use is a simple rule. Round-Robin is easy to understand: send one task to worker 1, the next to worker 2, and so on. This is useful as a baseline, but it ignores whether the worker has enough resources, whether it is already busy, and whether the task is likely to miss its deadline.

Heuristic schedulers improve this by adding rules. For example, a scheduler can prefer the worker with the most available CPU or the lowest estimated risk. However, fixed rules are difficult to tune because the right trade-off changes with workload type. A CPU-heavy task, a memory-heavy task, and a burst of short tasks do not need the same decision rule.

This is why reinforcement learning was explored. In reinforcement learning, an agent learns by taking actions and receiving rewards [1]. For cluster scheduling, the state is the current cluster condition, the action is the selected worker, and the reward measures whether the placement was useful. This matches the nature of scheduling: every placement changes the next state of the cluster.

1.3 | Project Objectives

The objectives of the project were:

- Build a working Go master-worker cluster that can run Docker-based tasks.
- Maintain worker registration, heartbeat, resource tracking, task lifecycle, and result storage.
- Provide a clean scheduling boundary so different scheduling algorithms can be compared.
- Implement baseline scheduling and PPO-based scheduling.
- Train PPO using realistic cluster traces and a reward function aligned with feasibility, queue pressure, tail pressure, and load balance.
- Evaluate the system with repeatable campaigns and explain the results honestly.

1.4 | Scope

The core scope is the distributed cluster and PPO scheduler. The report intentionally does not focus on auxiliary interfaces or monitoring tools. They may exist as supporting utilities in the repository, but the technical contribution is the Go control plane, worker execution layer, and reinforcement learning scheduler.

1.5 | Main Contributions

The main contributions are:

1. A Go-based distributed task execution framework with master and worker nodes.
2. A scheduler interface that supports simple baselines and PPO scheduling.

3. A PPO service that receives scheduling requests over gRPC and returns a worker decision.
4. A trace replay training environment with lifecycle-based resource accounting.
5. A reward design that penalizes infeasible placement, queue pressure, tail pressure, and imbalance.
6. A benchmark setup for comparing scheduling behavior across load patterns.

Chapter 2

Literature Survey

2.1 | Related Work

2.1.1 | Distributed Task Scheduling

Distributed systems divide work among several machines. The scheduler in a cluster chooses the location where every task is to be executed. The implications of this decision are on completion time, resource utilization, fairness, and recovery after failure. A scheduler also has to respond to the imperfect information, since the state of the workers is changing as the tasks are being submitted.

2.1.2 | Reinforcement Learning to Scheduling

Reinforcement learning is the study of how an agent may learn to make decisions by acting on an environment and receiving feedback signals of reinforcement of its actions onto an environment. A typical RL configuration has:

- **State**: what the agent can see.
- **Action**: what the agent has a choice of.
- **Reward**: feedback post action.
- **Policy**: the rule or model upon which actions are chosen.

In this project, the mapping is straightforward. The task and workers that are present are the state. The action is the selected worker. The reward is a quality of placement. Scheduler is the policy.

2.1.3 | Q-Learning and Q-Tables

Q-learning is an example of a classic reinforcement learning algorithm proposed by Watkins and Dayan [2]. It learns a value, referred to as the $Q(s, a)$ value meaning the expected

long-term reward of taking action a in state s. These values can be stored in a table in a small environment.

The appeal of a Q-table lies in the simplicity of the Q-table. When there are few states and actions, all the state-action pairs can be stored in the table. However, cluster scheduling does not have a small state space. The combination of CPU, memory, storage, task type, queue depth, workers availability and workers load can create a large number of possible states. This would either make a table enormous or necessitate violent discretization, which would be losing valuable information.

2.1.4 | Deep Q-Networks

Deep Q-Networks (DQN) are the replacement of Q-table with a neural network. Mnih et al. demonstrated that deep neural networks can approximate action-value functions when the inputs are of high dimension size [3]. This addresses the storage issue of Q-tables since the model generalizes the similar states.

DQN is a natural next step after Q-tables, but it is still not ideal for this project. DQN estimates a value for each discrete action. In cluster scheduling, the number of candidate workers can change, workers can become infeasible, and invalid actions must be masked. DQN can be adapted, but the implementation becomes less natural when the action set is dynamic and the objective is to improve a stochastic placement policy.

2.1.5 | Proximal Policy Optimization

Proximal Policy Optimization (PPO) is a policy-gradient algorithm proposed by Schulman et al. [4]. Instead of only learning a Q-value, PPO directly learns a policy that maps states to action probabilities. It also learns a value function to estimate future return. The key idea in PPO is to avoid changing the policy too aggressively by using a clipped objective.

PPO was selected because it fits the scheduling problem better than a Q-table or plain DQN:

- It directly learns a worker-selection policy.
- It has the ability to make use of action masks in order to shun infeasible workers.
- It allows stable updates, using clipping.
- It is designed to co-exist with continuous states attributes like resource ratios.
- It enables offline training and optional online adaptation.

2.1.6 | Cluster Trace-Driven Evaluation

The scheduler can be rendered unrealistic by training him only on random synthetic tasks. The Alibaba cluster trace trace is a production cluster workload data that has been published to cluster management research [5]. In this project, Alibaba trace replay was used to subject the PPO scheduler to realistic patterns of task resource utilization and arrival characteristics.

2.1.7 | Modern Industry Standards and AI-Driven Trends

Within the present-day technological context, Kubernetes (K8s) has become the industry standard in terms of container orchestration [6]. Google had a heavily influential role in Kubernetes; internal cluster manager Borg was the inspiration, and the default scheduler strongly relies on heuristics: initially, it filters to find viable nodes; then it scores them based on resource availability and affinity rules. Although highly robust, these heuristics tend to have issues with long-term optimization and complex and non-linear resource relationships, which RL-based methods seek to address.

Besides general-purpose orchestration, there are specialized projects such as **Volcano** [7] and **Apache Yunikorn** [8] that have been created to support high-performance batch workloads and AI/ML training jobs on Kubernetes. These systems bring in advanced queuing and gang scheduling which are essential in the distributed training.

The emergence of both **Ray** [9] and **Dask** has also shifted the focus onto task-parallel computing, in this case, to Python-centric AI workloads. These frameworks offer top-down application-level scheduling, in contrast to Kubernetes which is often viewed as a bottom-up infrastructure manager.

Most recent developments in the field of **AIOps** and autonomous infrastructure are focused on the shift towards systems that are Agentic and autonomous in nature. In such systems, autonomous agents utilize machine learning-often based on variants of Reinforcement Learning such as PPO [4] or Soft Actor-Critic (SAC) [10]-to perform proactive failure recovery, dynamic auto-scaling and predictive load balancing. This project will be in line with these trends by having a custom Go-based framework that will incorporate a PPO agent straight into the scheduling loop, rather than having a centralized set of heuristics.

2.2 | Tools and Technologies

2.2.1 | Docker-based Execution

Container-based execution is a viable approach to portability of tasks. Docker takes an application and its dependencies and packages them into a container image, which can then be run by the worker node in an environment that is repeatable [11]. In this project, the worker node involves Docker as the boundary of execution and the master node stays in charge of making decisions regarding placement.

2.2.2 | Go for Cluster Control Plane

Go was chosen as the master-worker model as it is highly compatible with networked systems. It has built-in goroutines and channels are supported with concurrency, a large standard library, and performance compiled executables. According to the official documentation of Go, the language is an open-source project aimed at making a programmer more productive [12]. Go is used in the master server, worker server, task handling, scheduler interface and worker communication logic in this project.

2.2.3 | gRPC and Protocol Buffers

gRPC is used by the master and a worker. gRPC is a high-performance Remote Procedure Call framework which uses Protocol Buffers as service definitions and structured messages [13]. This renders it viable to the project since the master requires strictly defined calls like registering workers, assigning them tasks, heartbeat reporting, log streaming, and PPO worker selection.

2.2.4 | Persistence Layer

The implementation of task execution systems must be persistent since a task has a lifecycle: created, queued, assigned, running, completed, failed, or retried. The state of tasks, worker metadata, results, and scheduler model records were stored using MongoDB. MongoDB is document-based (records) that are represented by field-value pairs [14], which is useful in the case of task and worker objects, which may change during development.

Chapter 3

Methodology

3.1 | System Design

3.1.1 | High-Level Architecture

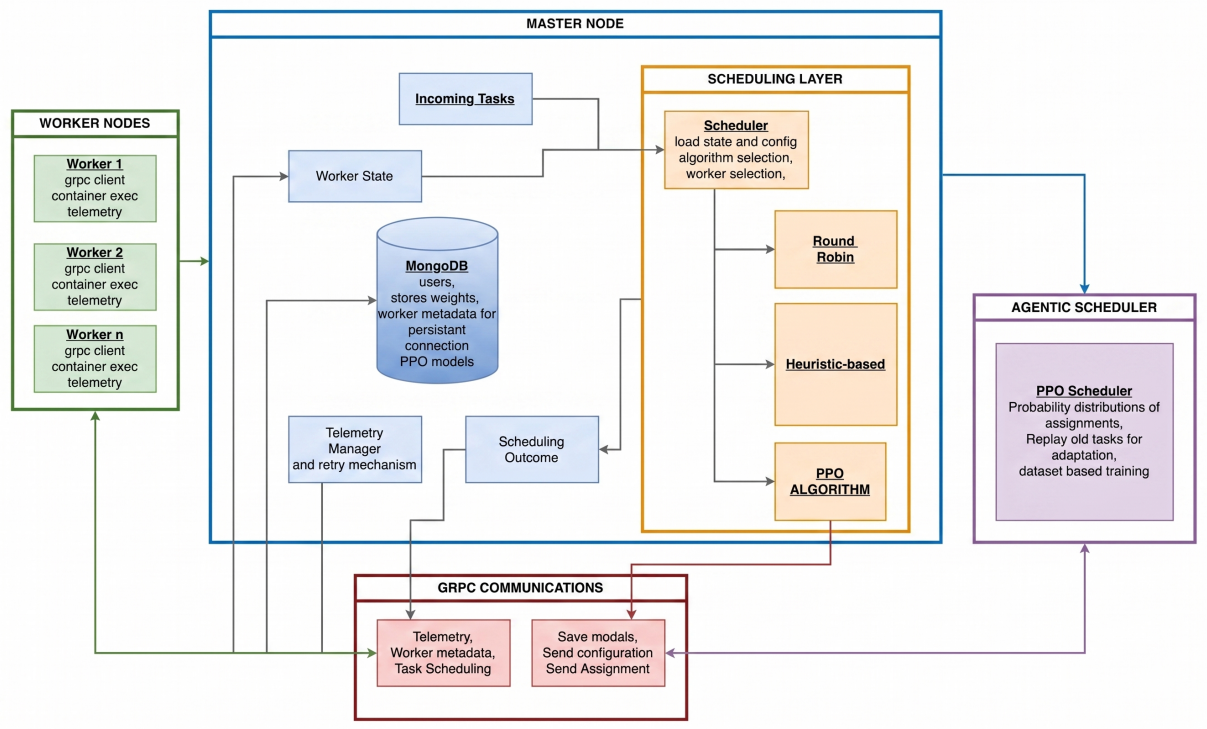

Figure 3.1.1: Overall architecture of Agentic Cloud Cluster

Description of Figure 1, the overall architecture: This system is organized using a master-worker model. Task submissions go to the master node which stores metadata, tracks workers, and calls the scheduler. The worker nodes execute the Docker tasks allocated to them and report back to the master. The PPO scheduler is decoupled as a Python service allowing the Go control plane to remain stable as the learning model is able to evolve independently.

3.1.2 | Go Distributed Cluster Architecture

The distributed cluster is achieved by a control plane in Go. Its design is based on a simple rule: the master decides to coordinate and the workers perform the work. This maintains the system easy to reason about. It is not necessary that a worker should know about other workers. It simply has to accept assignments of the master, execute the container and report health and task status.

At runtime, the cluster has four main layers:

1. **Client and API layer**: this layer accepts a task submission and management commands.
2. **Master control plane**: validates requests, stores task state, tracks workers, and picks a scheduler.
3. **Scheduler layer**: selects a worker by Round-Robin, RTS or PPO.
4. **Worker execution layer**: executes Docker containers and returns results.

GRPC is used to communicate between master and workers. Request and response formats get declared in Protocol Buffer files, and makes communication rigid and typed. Persistent storage is also used by the master to store records of tasks, worker records, task attempt records and result metadata records. The design consideration here is that scheduling, execution, and persistence are independent modules. This facilitated easier debugging of the project since a scheduler bug would not necessitate the re-writing of the worker executor, and a bug in worker execution would not require the PPO model to be changed.

3.1.3 | Master Node

The coordinator is the master node. Its responsibilities are:

- Accept task submissions.
- Keep worker registry and worker health.
- Track available CPU, memory and storage.
- Invoke the active scheduler on each of the tasks.
- Assign workers to do tasks.
- Store task state, attempts and results.
- Requeue tasks in case of failure by workers or when resources are not available.

On the inside, the master has a couple of significant modules:

Table 3.1.1: Master node modules

Module	Role
Server layer	Exposes gRPC calls being used by workers to register, heartbeat, task status, and report results.
Task handling	Generates tasks, records state transitions, queues tasks when workers are offline, and records attempts.
Registry of workers	Holds up-to-date worker identities, addresses, resource capacities, and current availability.
Scheduler package	Provides a common interface to Round-Robin, RTS, as well as PPO scheduling.
Persistence layer	Stores workers, tasks, assignments, attempts, and results.
Telemetry manager	Maintains the recent state of health and resources of workers to schedule and inspect the operators.

Of particular importance is the scheduler boundary. The master does not even have to be aware of the internal algorithm. It simply poses: given this task and these workers, which worker is to be given the task? The scheduler will reply with a worker ID and the master will verify that the worker has not disappeared yet and has sufficient resources before dispatching the task.

3.1.4 | Worker Node

The worker node has the role of carrying out tasks. Every worker opens a gRPC server, accepts orders made by the master, launches a Docker container, monitors the execution, streams logs, and reports final status. The worker also transmits heartbeat information allowing the master to have a live view of cluster resources.

The worker is responsible of three things:

- **Registration and heartbeat**: after registration, the worker periodically reports resource and status changes to the master.
- **Container execution**: the worker pulls or uses the requested Docker image, creates the container with the requested resource limits, runs it, and monitors the exit code.
- **Result reporting**: Once the container has left the worker reports success or failure and posts any output files produced by the task.

This makes the worker mostly stateless from a cluster point of view. When one worker unintentionally vanishes, the master will notice the missed heartbeats, and is also capable of requeuing tasks. The worker does not liaise with other workers, preventing the distributed locking of a complex nature.

3.1.5 | Control Flow in the Go Cluster

Normal task flow is:

1. A task is posted with image name, command and resource requirements.
2. The master puts the task in created or queued state.
3. The master develops a live picture of workable workers.
4. The active scheduler picks a worker.
5. The master articulates the task to such a worker using gRPC.
6. The worker executes the Docker container and logs or streams logs.
7. The worker gives a report on completion.
8. The result is stored in the master which emancipates the worker resources.

The system also promotes recovery behavior. In case a worker is lost, the master will record the attempt as lost and reenact the logical task. What is significant about this separation between a logical task and an attempt to execute the task is that a task may require many more than one attempt before it reaches a final state.

3.1.6 | Resource Accounting

Resource accounting is concerned with scheduling. Every employee promotes a sum of the CPU, memory, and storage. The master tracks allow available resources to be tracked by the difference between the resources used by the tasks that are currently being assigned. As soon as a piece of work is done, these resources are freed. The simple types of schedulers use this accounting, as well as the action mask of PPO.

The design does not provide tasks to workers that they cannot fit in. Although PPO may be true about a worker, the Go master will still confirm the decision prior to assignment. This safeguards the cluster against stale model output or stale worker state.

3.1.7 | Task Lifecycle

Tasks are first in the master queue and stay in the queue until at least one worker is available to meet the resource constraints. When feasible worker is available, the scheduler picks a target and the master picks an execution attempt. In case of dispatch failure because of stale state or connectivity failures, the attempt is marked lost and the logical task is returned to the queue to go through another scheduling cycle.

Once it has begun to execute, the worker sends back to the master status of completion. Complete runs are recorded as complete. Recoverable and non-recoverable: Failed runs are classified as recoverable or non-recoverable, recoverable failures are retried using requeueing, and non-recoverable failures terminate the task as failed.

3.1.8 | Task Execution Flow

The definite path of execution begins at the API where the task is tested and stored and then transferred to the master queue. The queue processor constructs a new worker snapshot, calls the active scheduler, and checks the viability of the selected worker before reserving resources and sending an `AssignTask` call.

When the worker accepts and initiates the container, the effort is continued until the end and the master maintains the end result and releases reserved resources. In case of assignment or execution failure caused by transient failures (such as loss of workers), the attempt is marked lost and the logical task is requeued to reschedule it. It is this attempt-level recovery that allows safe retries without any loss in task-level traceability.

3.1.9 | Scheduler Interface

The scheduler receives:

- task CPU requirement,
- task memory requirement,
- store requirement,
- task type,
- worker capacity,
- worker availability,
- worker activity status.

It yields a worker ID. This easy interface enabled the project to begin with baselines and subsequently incorporate PPO. The PPO scheduler connects to a Python service using gRPC. When the PPO service is not available or has an inappropriate worker, the Go scheduler will revert to a safer scheduler. This was an intentional design choice since a learning model is expected to enhance scheduling, and not render the entire cluster vulnerable.

3.1.10 | PPO Deployment Design

The PPO deployment is in support of three modes:

Table 3.1.2: PPO deployment modes

Mode	Meaning
Active	PPO makes the scheduling decision and falls back on failure.
Shadow	PPO is queried for comparison, but the fallback scheduler is used for the actual placement.
Fallback	PPO is not used but the fallback scheduler is invoked directly.

This design was selected since the deployment of models in systems work should be conservative. The system is able to test the PPO decisions without necessarily believing them in the production paths.

3.2 | Why Reinforcement Learning for Scheduling

Scheduling is a repeated decision problem. Each task placement affects the future state of the cluster. If a large task is placed on the only high-memory worker, a later memory-heavy task may fail or wait. If all tasks are sent evenly without checking resource demand, a weak worker may become overloaded. This is why the scheduler should learn from consequences, not only follow a fixed rule.

In reinforcement learning terms:

- **Agent**: the scheduler.
- **Environment**: the cluster and incoming task stream.
- **State**: task features and worker resource features.
- **Action**: select one worker.
- **Reward**: score the placement based on feasibility, delay risk, and balance.

3.3 | Starting Point: Q-Tables

The simplest RL idea is a Q-table. It stores a value for every state-action pair:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Here, s is the current state, a is the selected action, r is the reward, s' is the next state, α is the learning rate, and γ discounts future reward.

For a small example, this works nicely. Suppose there are only two workers and three task types. A table can store the value of placing each task type on each worker. But the real cluster state is much larger. A worker is not just “free” or “busy”. It has CPU, memory, storage, current load, and active status. A task also has CPU, memory, storage, task type, and SLA multiplier. If each feature is discretized into even a few buckets, the number of table rows grows very quickly.

Decision: Q-tables were useful for understanding the RL formulation, but not suitable for implementation because the state space is too large and continuous.

3.4 | Next Step: Deep Q-Networks

DQN solves the table-size problem by using a neural network to approximate Q-values:

$$Q(s, a; \theta) \approx \text{expected future reward}$$

This is better because the network can generalize. If it has seen a worker at 60 percent memory usage, it can still make a reasonable prediction for 62 percent memory usage.

However, DQN has limitations for this project:

- The action set depends on the number of workers.
- Some workers are invalid for a task because they do not have enough resources.
- The scheduler needs a clean way to mask invalid actions.
- We wanted a direct policy over workers, not only a value estimate.
- Stable DQN training usually requires replay buffers and target networks, which adds complexity when outcomes are delayed.

Decision: DQN was a logical next idea after Q-tables, but PPO was selected because policy-gradient learning matches the worker-selection action more naturally.

3.5 | Final Choice: PPO

PPO directly learns a policy $\pi_\theta(a|s)$, which means the probability of choosing worker a in state s. It also learns a value function $V_\theta(s)$, which estimates how good the current state is.

The main PPO objective used is the clipped surrogate objective:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right]$$

where:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta old}(a_t|s_t)}$$

$\hat{A}_t$ is the advantage estimate. It tells whether an action was better or worse than expected. The clipping term prevents the new policy from moving too far away from the old policy in one update. In this project, this stability was important because bad scheduler updates can directly affect task placement.

3.6 | PPO Architecture

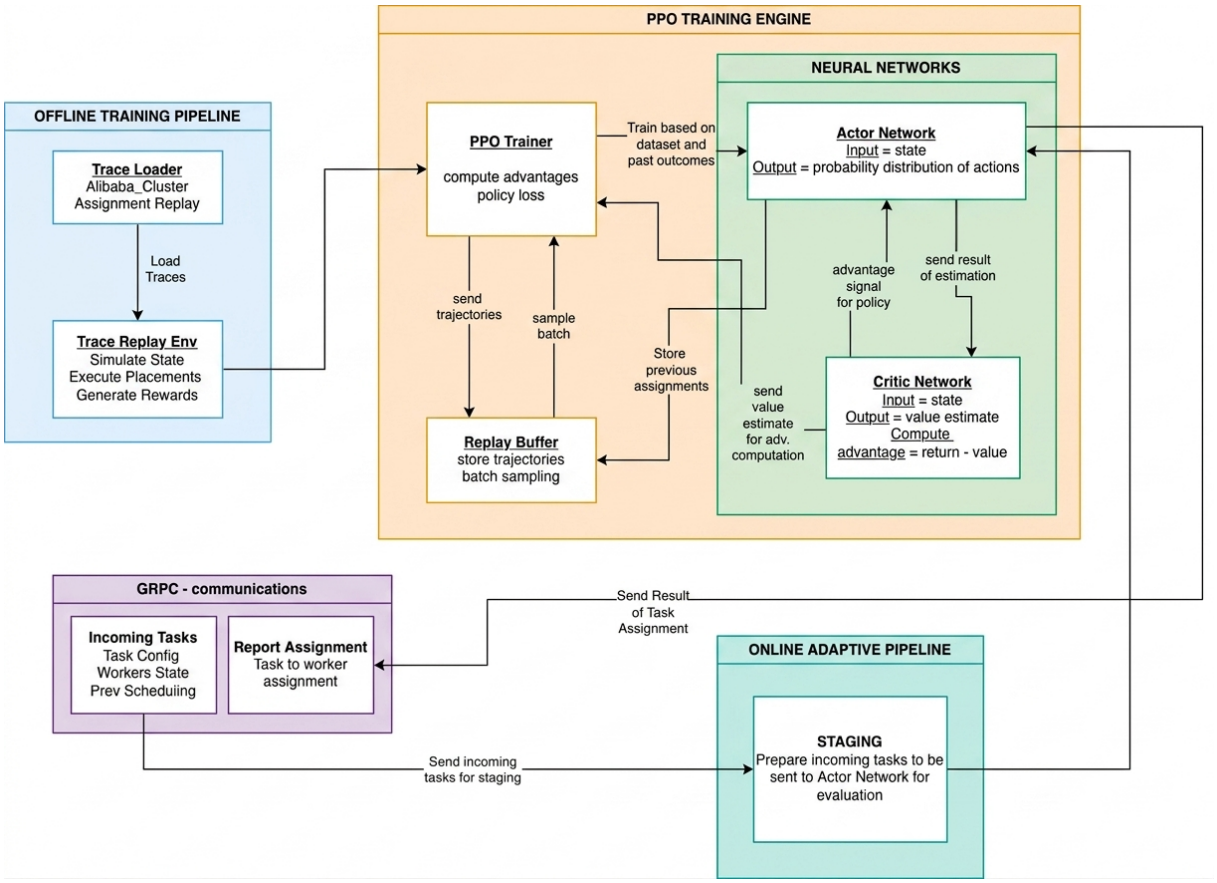

Figure 3.6.1: PPO scheduler architecture

Explanation of Figure 3.6.1: The PPO scheduler receives task features and worker features. These are encoded into pairwise task-worker inputs. The neural network produces policy logits for worker selection and a value estimate for the state. The policy head decides which worker should receive the task. The value head helps calculate advantages during training. The Go master calls the Python PPO service through gRPC, while the Python service loads the trained PyTorch checkpoint.

3.7 | Actor and Critic Network

The PPO model is an actor-critic network. Both the actor and critic share the same pairwise encoder. For each scheduling request, the task vector is repeated once for every candidate worker and concatenated with that worker's feature vector. This creates one task-worker row per possible action.

The shared encoder has two fully connected layers with ReLU activation:

$$\text{task-worker features} \rightarrow \text{Dense}(128) \rightarrow \text{ReLU} \rightarrow \text{Dense}(128) \rightarrow \text{ReLU}$$

The actor-critic data path is shown in Figure 3.7.1, rendered directly from the project Mermaid specification.

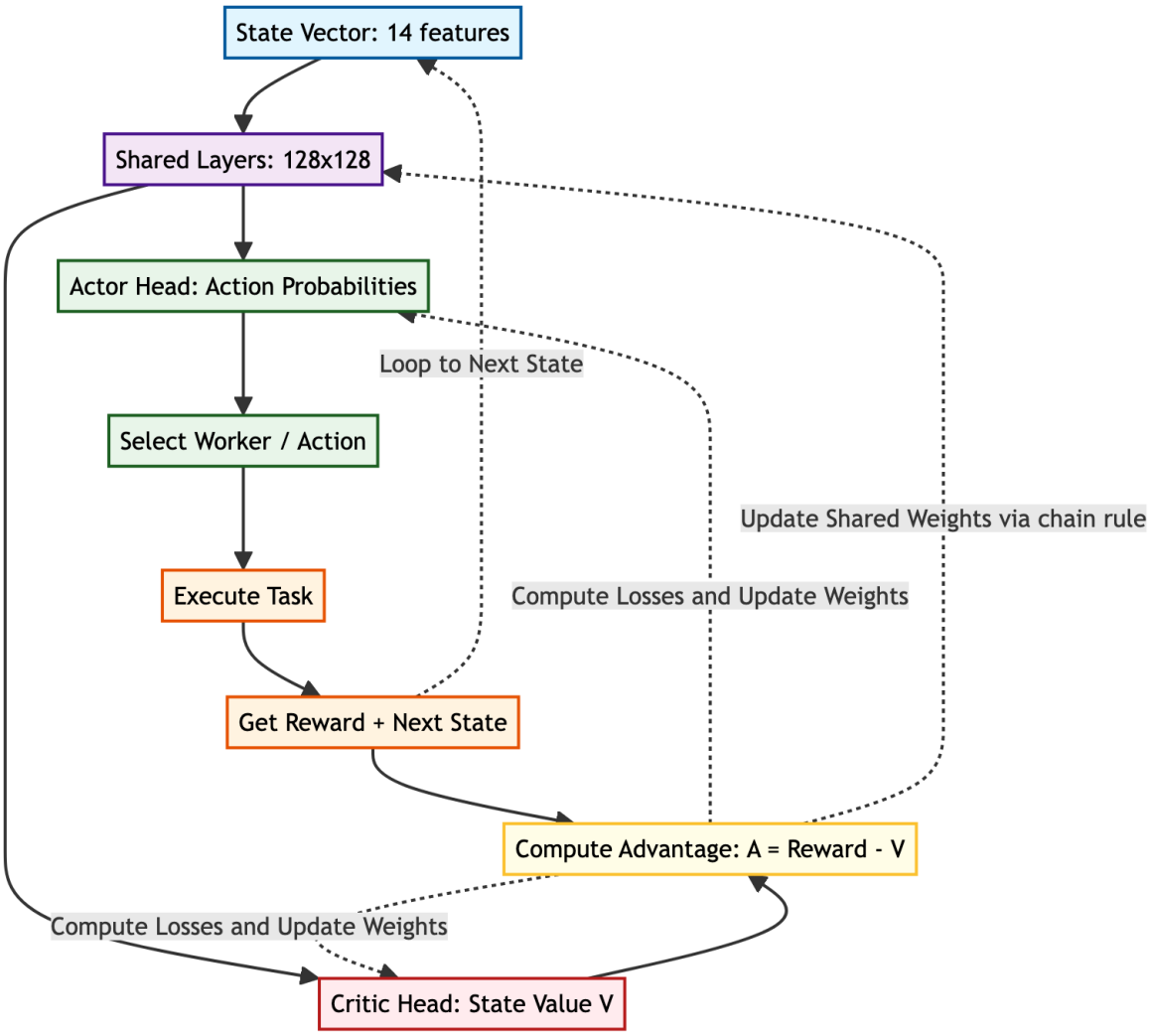

Figure 3.7.1: Actor-critic data flow and PPO learning feedback loop

After the shared encoder, the network splits into two heads:

- **Actor / policy head**: produces one logit per worker. After masking infeasible workers, these logits define the probability distribution $\pi_\theta(a|s)$.
- **Critic / value head**: produces a single value estimate $V_\theta(s)$, which predicts the expected future reward from the current cluster state.

The critic is not choosing the worker. Its job is to judge the state. During training, it answers the question: "From this task and current worker set, how much reward should the scheduler expect on average?" PPO then compares the actual reward to this expectation. If the selected action performs better than the critic expected, the advantage is positive and PPO increases the probability of similar choices. If it performs worse, the advantage is negative and PPO reduces that probability.

Formally, the critic estimates discounted return:

$$V_\theta(s_t) = \mathbb{E}_{\pi_\theta}\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \,\middle|\, s_t\right].$$

In this scheduler, $V_\theta(s)$ is not a probability and does not have a fixed $[0, 1]$ range.

Its size is based on the reward design and discount horizon. Single-step rewards are typically around a limited practical band (such as infeasible placements receive a reward of -1.8), with feasible low-pressure placements being allowed to remain positive because of the terms of 1.4 + 0.25H), but queue and tail-pressure penalties can push rewards down during congestion. Thus, in healthy and balanced conditions, under normal conditions, it is assumed that the equation V 0 -s (s) is positive, and under the conditions of stress, frequent requeues, or improper placement, the equation V 0 -s (s) can have a negative value. Effective horizon (which is approximately equal to 1 divided by 1- gamma)) increases magnitude since it is cumulative discounted reward.

In implementation, the value head is used following a pooling of the encoded worker representations. This implies that the critic will be viewing the entire set of candidates and not just the worker who has been chosen. This is important since a scheduling state is determined by the combination of all the available workers. A worker has 3 GB of free memory; it is good or bad depending on what the other workers look like.

The critic is trained in the form of a clipped loss of values:

$$L^{VF} = \max\left((V_\theta - V_{target})^2,\ (V_{clipped} - V_{target})^2\right)$$

In which case, V clipped is a limit to the extent to which the value estimate can vary across one update. This is a reflection of the conservative policy update by PPO. It does not allow the critic to suddenly shift its estimate too far which would render advantage estimates unstable.

The target value is borrowed out of the rollout return, it is not obtained by a fixed constant. Using standard PPO with GAE, we first compute

$$\delta_t = r_t + \gamma V_{\theta old}(s_{t+1}) - V_{\theta old}(s_t), \qquad \hat{A}_t = \sum_{l=0}^{T-1} (\gamma\lambda)^l \delta_{t+l}.$$

Then the target of the critic is.

$$V_{target,t} = \hat{A}_t + V_{\theta old}(s_t),$$

that is the bootstrapped return of the state at the parameter of that state, namely, 6.3. When a trajectory segment terminates in a terminal state, the bootstrap term in terms of which it appears, vold(s t + 1): is dropped (set to zero).

The total PPO loss is the policy loss, plus the value loss, and an entropy term:

$$L = L^{CLIP} + c_1 L^{VF} - c_2 H[\pi_\theta]$$

As an example of a categorical policy over a set of workers, entropy is:

$$H[\pi_\theta(\cdot|s)] = -\sum_{a=1}^{N_w} \pi_\theta(a|s) \log \pi_\theta(a|s),$$

whereby N_w is the number of practicable worker actions following mask and that $\pi_\theta(a|s)$ is derived off the masked softmax probabilities. High entropy implies that the policy is distributed among a large number of workers (exploration), and low entropy implies that the policy is concentrated on a small number of workers (near-deterministic behavior). In this project, critic learning is weighted by the value coefficient c_1 and the entropy coefficient c_2 prevents premature determinism where the scheduler is repeatedly tempted to select one apparently strong worker too early. With more training and a more calibrated model, the entropy naturally decreases.

3.8 | State Representation

The PPO state is constructed based on the features of task and worker. There are five values in the task vector:

Table 3.8.1: Task feature vector

Feature	Meaning
Required CPU	CPU requested by the task.
Required memory	Memory requested by the task.
Required storage	Storage requested by the task.
SLA multiplier	How much delay is acceptable compared to task runtime.
Task type	Encoded task category, like CPU-intensive, memory-intensive or mixed.

Each worker is represented using nine values:

Table 3.8.2: Worker feature vector

Feature	Meaning
available cpu ratio	available CPU divided by total CPU.
Available memory ratio	Available memory divided by total memory.
Available storage ratio	Available storage divided by total storage.
Total CPU	Absolute CPU capacity.
Total memory	Absolute memory capacity.
Total storage	Absolute storage capacity.
Used CPU ratio	Used CPU/total CPU.
Used memory ratio	Used memory divided by total memory.
Used storage ratio	Used storage/total storage.

The model uses pairwise task-worker features. This is significant since it is not only what is the task? or what is the worker? but how suitable is this worker to this task?

3.9 | Action Space and Masking

The action is the index of the worker that is selected. The system develops an action mask before PPO makes a choice. Marking a worker feasible is only possible when it is active and has sufficient CPU, memory, and storage to carry out the task. The infeasible workers are disguised in such a way that the model does not have the option of picking the infeasible workers during normal operation.

3.10 | Reward Function

The reward was to teach PPO how to behave in practice, that is, to schedule:

$$R = 1.4 + 0.25H - 0.35Q_p - 0.55T_p - 0.20I_p - 0.40\Delta I - R_q$$

where:

- H is headroom bonus. It is rewarded by the selection of workers who have space vacant.
- Q_p is queue pressure. It punishes anticipated waiting.
- T_p is tail pressure. It penalizes likely SLA or tail-latency risk.
- I_p is imbalance penalty. It does not encourage overworking of already overworked workers.
- ΔI is change in imbalance. It punishes those behaviors that worsen cluster balance.
- R_q is requeue penalty. It discourages constantly dangerous placements.

The values are calculated with regard to the projected state upon the task being placed on the chosen worker. To begin with the system estimates the worker load as an average of the CPU, memory, and storage usage:

$$L_w = \frac{1}{3}\left(\frac{\text{usedCPU}_w}{\text{totalCPU}_w} + \frac{\text{usedMem}_w}{\text{totalMem}_w} + \frac{\text{usedStorage}_w}{\text{totalStorage}_w}\right).$$

Once the requested resources of the new task are added to the available worker, it then becomes the projected load $L_{selected}$. The average load in the cluster is:

$$L_{cluster} = \frac{1}{N}\sum_{w=1}^{N} L_w$$

Based on these values, the terms of the rewards are:

Table 3.10.1: Reward term computation

Term	How it is computed and what it means
H	$1 - L_{selected}$. A lightly loaded selected worker gives a higher headroom bonus. If the worker is almost full, this value becomes small.
Q_p	$\min(queueWaitProxy/slaBudget, 3.0)$. The proxy is trace queue wait plus $runtime \times L_{selected}$. It estimates how much waiting pressure the placement creates relative to the SLA budget.
T_p	$\max(turnaroundProxy/slaBudget - 1, 0)$. Turnaround proxy is queue wait proxy plus runtime. This term becomes positive only when expected turnaround crosses the SLA budget.
I_p	$\max(L_{selected} - L_{cluster}, 0)$. This is imbalance in the system. It means the selected worker is more loaded than the cluster average. If the selected worker is below or equal to average load, the penalty is zero.
ΔI	$\sigma(loads_{after}) - \sigma(loads_{before})$. This measures change in imbalance using the standard deviation of worker loads. Positive means the action made the cluster less balanced. Negative means it made the cluster more balanced.
R_q	$0.05 \times \min(requeueCount, 4)$. A task that has already been requeued receives a small extra penalty so the scheduler avoids repeating risky placements.

For example, if one worker is already much busier than the others, its L_w will be above $L_{cluster}$. Placing another task there increases I_p, and it may also increase ΔI if the spread between worker loads becomes larger. In system terms, imbalance means a hot-spot: one worker is carrying more active resource load while other workers still have useful capacity. The reward therefore teaches PPO to avoid placing every task on the same apparently strong worker.

If a worker is infeasible, the reward is -1.8. This strong negative reward teaches the policy that violating hard resource limits is worse than merely making a slow but feasible placement.

3.11 | Training Data

The PPO model was trained using trace replay. The Alibaba cluster trace was used because it contains real production workload behavior [5]. The trace replay environment presents tasks in chronological order. When a task is placed, its resources are reserved on the selected worker. When its simulated runtime ends, resources are released.

Decision: Lifecycle-based resource tracking was used instead of simple exponential decay. This matters because many trace arrivals are simultaneous. A decay model can release or retain resources incorrectly, while lifecycle tracking matches the actual running period of each simulated task.

3.12 | Training Pipeline

The training pipeline follows these steps:

1. Load trace tasks and worker information.
2. Normalize task and worker features.
3. Create the replay environment.
4. Run PPO rollouts and collect actions, rewards, log probabilities, and values.
5. Compute returns and advantages.
6. Update the policy using PPO clipping, value loss, entropy regularization, and gradient clipping.
7. Save model checkpoints.
8. Promote the best checkpoint for cluster use.

3.13 | Implementation in the Cluster

The Go master has a PPO scheduler wrapper. For each task, it builds a candidate worker list and sends it to the Python PPO service. The service encodes the request, applies the action mask, runs the policy, and returns the selected worker ID.

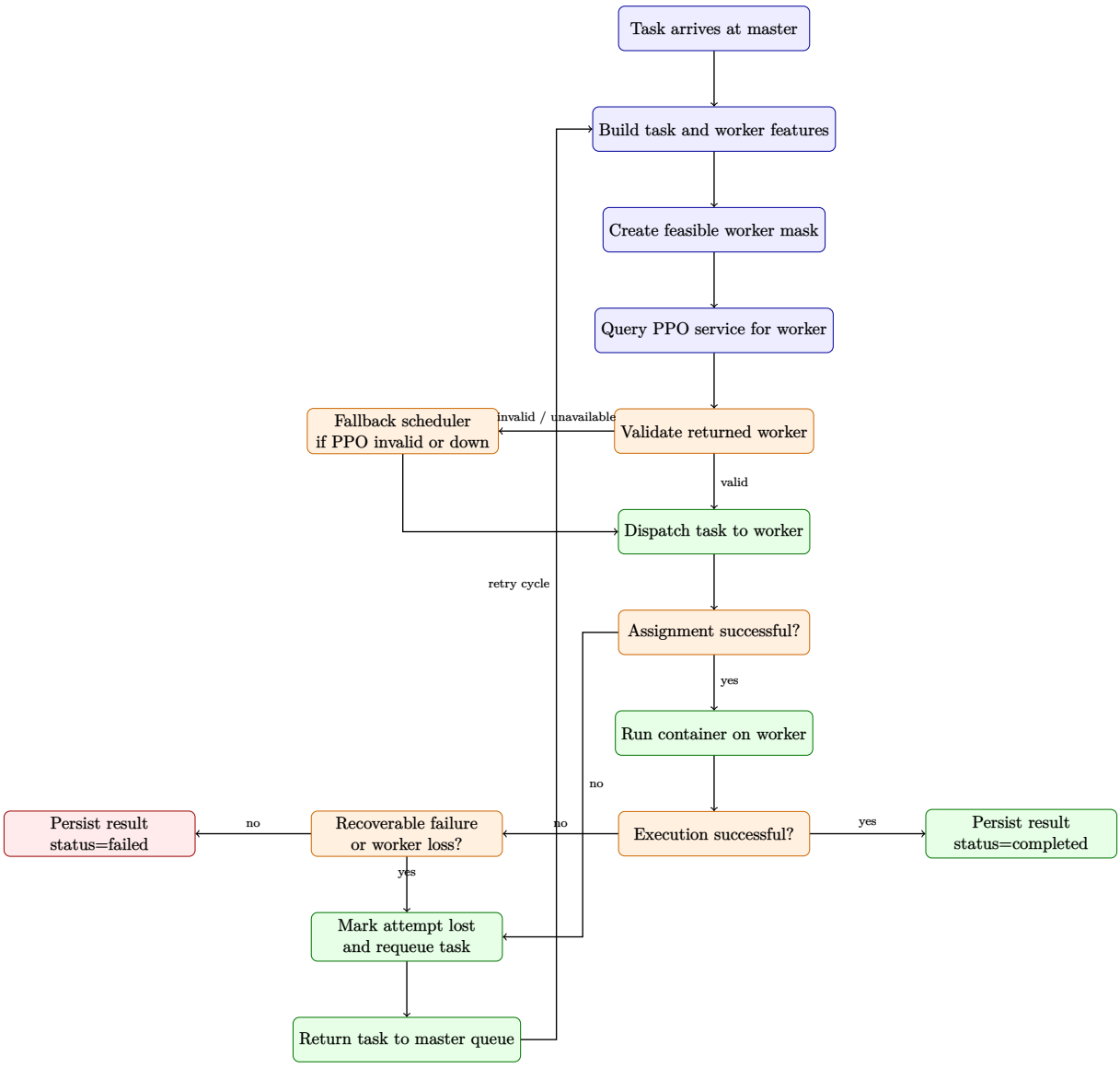

Figure 3.13.1: PPO scheduling flow inside the Go cluster with failover and requeue

Explanation of Figure 3.13.1: The flow starts in the Go master, not inside the neural network. After PPO returns a worker, the master validates that worker against the latest state. If PPO is unavailable or the choice is invalid, fallback scheduling is used. Assignment failures and recoverable runtime failures are not terminal: the current attempt is marked lost and the task is requeued for another scheduling cycle. Only non-recoverable execution failures are finalized as failed.

The implementation also includes fallback behavior:

- If the PPO service is down, use the fallback scheduler.
- If PPO returns an empty result, use fallback.
- If PPO returns a worker that no longer exists or cannot fit the task, use fallback.
- If shadow mode is enabled, log PPO's decision but execute fallback's decision.

Decision: The learning model is never allowed to be the only safety mechanism. The Go scheduler validates PPO's output before assigning a task.

Chapter 4

Results

4.1 | Project Outcomes

This chapter reports the measurable outcomes of the project and ties those outcomes to practical system behavior in the deployed cluster.

4.1.1 | Evaluation Setup

The system was evaluated in two ways:

1. **Offline PPO evaluation**: the trained model was tested on trace replay data to measure reward and feasibility.
2. **End-to-end cluster campaigns**: the Go master and workers executed real Docker tasks under different workload patterns and scheduler choices.

The main schedulers compared were:

Table 4.1.1: Schedulers compared

Scheduler	Description
Round-Robin	Simple cyclic assignment baseline.
RTS	Risk-aware heuristic scheduler using resource and risk scoring.
PPO	Reinforcement learning scheduler trained through trace replay.

4.1.2 | Training Convergence

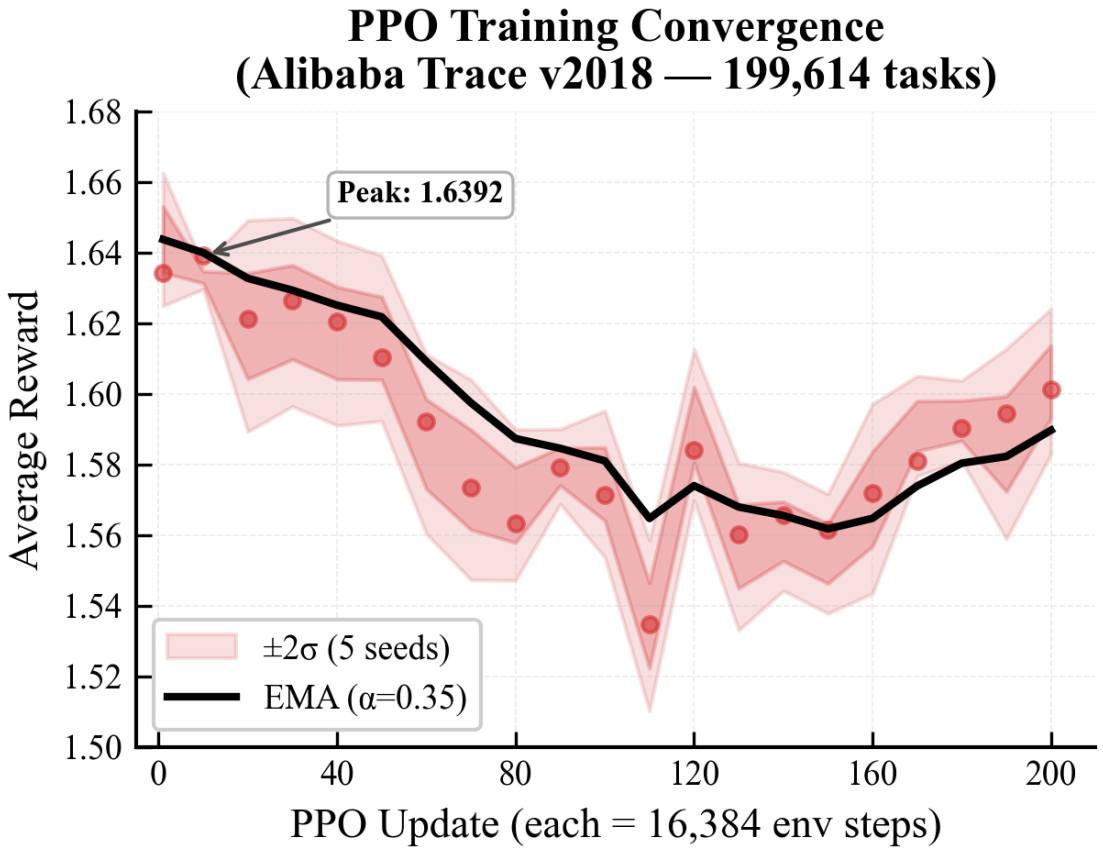

Figure 4.1.1: PPO training reward curve

Explanation of Figure 4.1.1: The curve shows how the average reward changed during training. Early training is exploratory because the policy is still testing placements. Later, the reward stabilizes, which indicates that the policy has learned a consistent placement strategy for the replayed cluster trace.

Average reward means the mean reward received per scheduling decision during a training window. From the DRL point of view, it is the signal that tells whether the actor is choosing actions that the environment considers better than before. A rising average reward means the policy is getting more positive feedback from the reward function. A flat reward means the policy has probably reached a stable behavior for the current trace and hyperparameters.

From the scheduling point of view, average reward is not a raw time measurement.

It is a composite measure of feasible practical placement quality: feasible placement, adequate worker headroom, reduced waiting pressure, reduced tail-risk pressure, and better cluster balance. Then when the average reward that PPO is learning to achieve is better, it does not imply that it is learning to make placements more likely to finish one task quickly, but rather to achieve a better average reward.

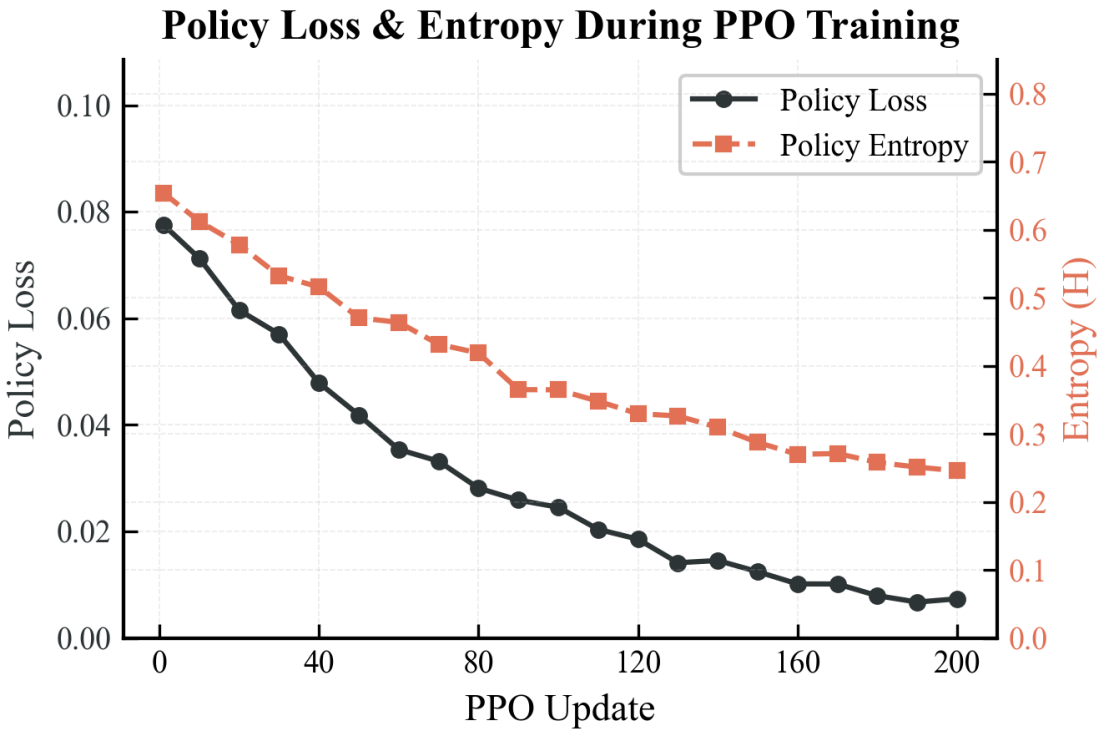

Figure 4.1.2: PPO policy loss and entropy

Explanation of Figure 4.1.2: Figure illustrates policy loss and entropy. The PPO actor loss is known as policy loss. It is a measure of the extent to which the policy is yet to change in order to increase the likelihood of high-advantage actions and reduce the likelihood of low-advantage actions. In simple terms, it indicates the amount of correction that the actor still requires. A diminishing policy loss implies the updates are becoming smaller since the scheduler is training on a more stable placement rule.

Entropy is used to quantify the randomness in the policy distribution. In the DRL perspective, high entropy implies that the agent is searching a large number of actions; low entropy implies that the agent is very certain and deterministic. The scheduling perspective of high entropy would imply that PPO is still experimenting with various employees to perform similar jobs. A smaller entropy implies that it has acquired more robust preferences, such as a preference towards a worker with greater resource fit and with less tail risk.

The loss of values is not represented in this figure since the chart was retained at two axes to be readable, but it is included in the PPO update. Value loss is part of the critic network. It is a metric of the error between the predicted state value, V(s), and the observed return due to the rollout. Under the point of view of scheduling, value loss informs whether the critic is learning to evaluate cluster states appropriately. When the loss of values decreases, then the critic is becoming more proficient in estimating whether the current task-worker situation will result in good future scheduling rewards.

4.2 | Results of the Offline PPO Model

The word trace in this section means a historical workload record. It holds the task arrivals, requested resources, run times and machine information. In trace replay, the environment submits these tasks to the scheduler in chronological order. The scheduler does not just copy the historical placement; it picks the workers in the simulated cluster, reserves the resources to the simulated runtime, releases the resources when the tasks finish, and is rewarded on each choice.

The PPO model that performed best scored a mean reward of about 0.8801 on the evaluation trace, and 0.8688 on the Round-Robin evaluation trace. The action rate of Round-Robin and PPO also improved, with Round-Robin having a better rate of 94.38

Table 4.2.1: Comparison of offline schedulers on trace replay

Policy	Mean Reward	Feasible Action Rate
Round-Robin	0.8688	94.38%
First feasible	0.8145	94.80%
Max available	0.8800	94.84%
PPO best model	**0.8801**	**94.86%**

The mean reward is obtained by dividing the value of rewards by all the scheduling choices made in the evaluation trace. It relies upon the same reward function as used in the methodology: feasibility, headroom, queue pressure, tail pressure, imbalance, change in imbalance, and requeue penalty. Calculation of feasible action rate is:

$$\text{Feasible Action Rate} = \frac{\text{number of chosen workers that meet the task resources}}{\text{total scheduling decisions}} \times 100$$

In scheduling theory Feasible action rate is a measure of the frequency with which the scheduler selected a worker capable of actually executing the task without breaking the CPU, memory or storage constraints. Light-load trace replay does not make the improvement enormously large, but it is a meaningful improvement since PPO has now reached the level of a strong heuristic without compromise of the ability to learn by reward signals.

4.2.1 | End-to-End KPI Comparison

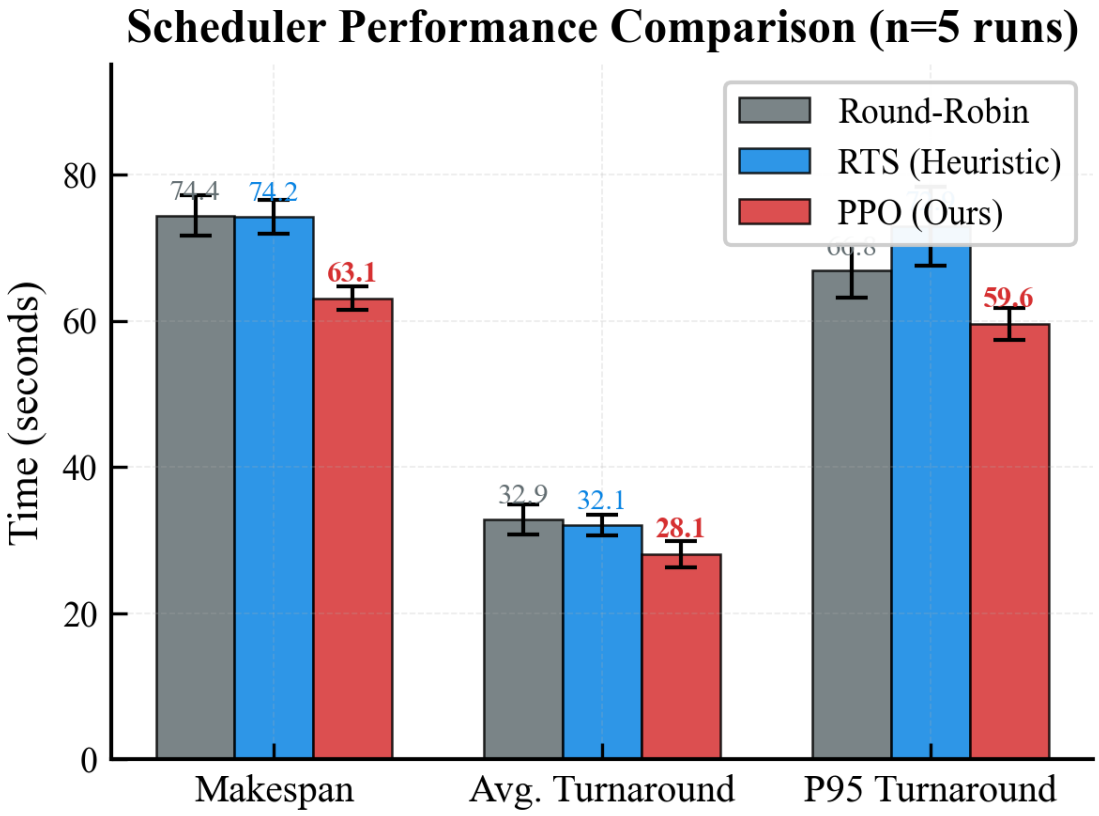

Figure 4.2.1: KPI comparison between schedulers.

Explanation of Figure 4.2.1: This plot will compare the key scheduler metrics. It is not important that PPO will score higher in every measure in every instance. The truth of the matter is that PPO will shine best when under stress, and more basic schedulers will be able to compete when the workload is not so demanding. This is a real systems outcome since introducing intelligence introduces overhead of inference.

4.2.2 | Multi-Run Comparison

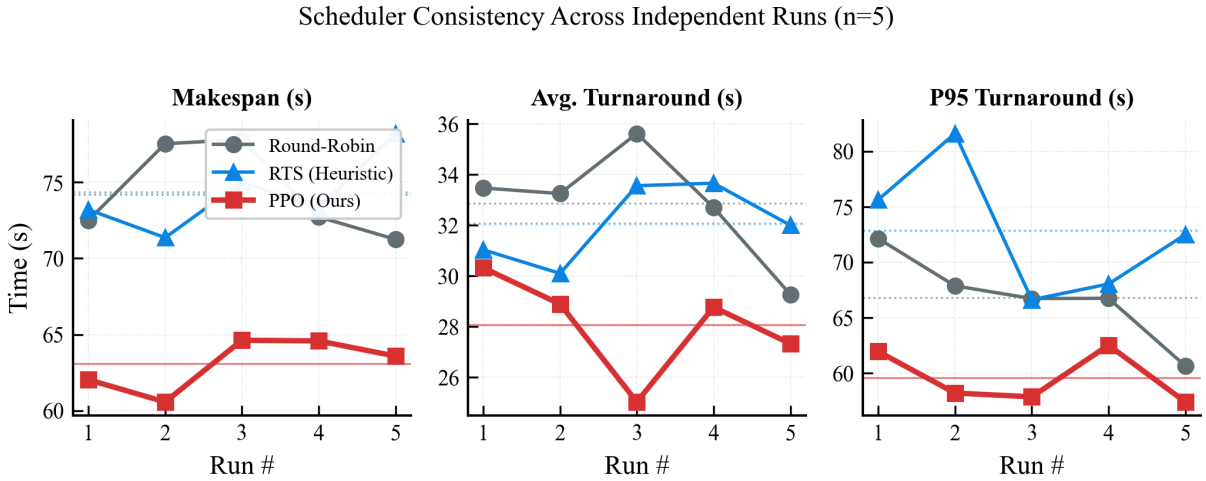

Figure 4.2.2: Multi-run scheduler comparison

Figure 1.1: Exploration: In a single run, timing of tasks, container start and worker state may be inaccurate. Multi-run plot is used to compare schedulers in the context of repeated runs. It indicates a consistent superiority of a scheduler or just a lucky event in a single campaign. This assisted in determining that the benefit of PPO is more evident under burst and overload conditions than under simple steady conditions.

4.2.3 | Cumulative Completion

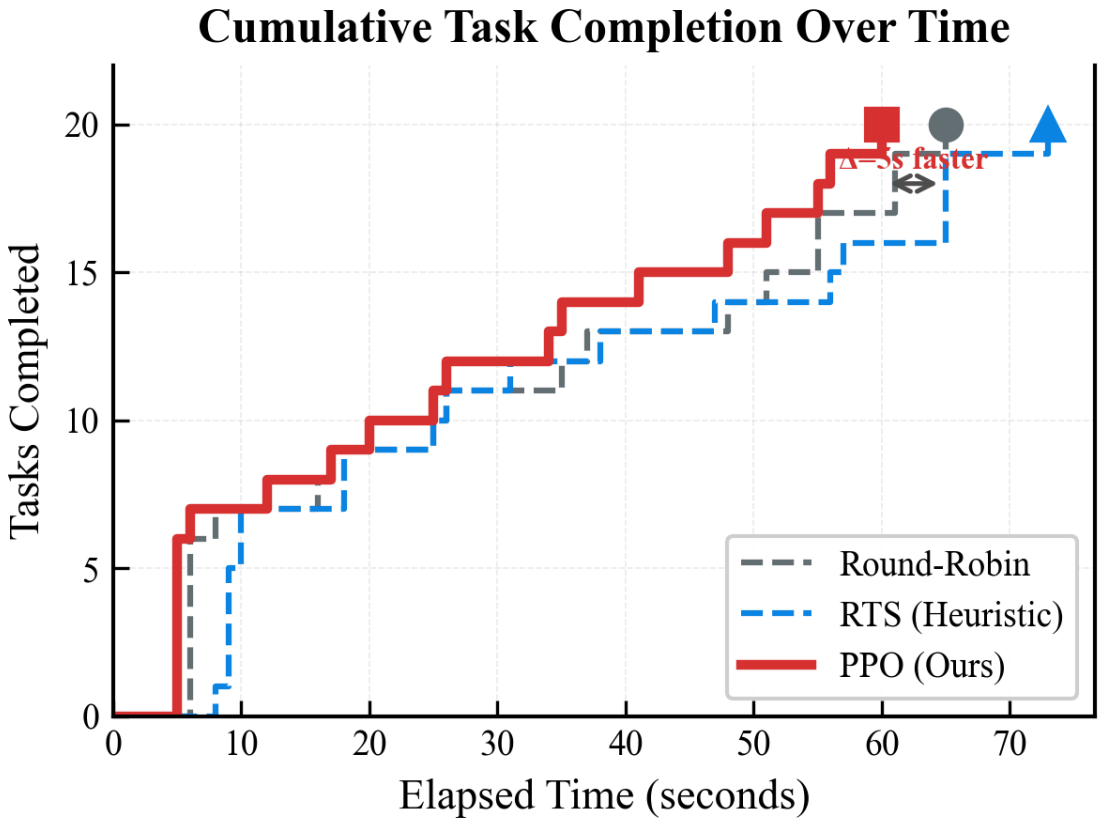

Figure 4.2.3: Cumulative number of tasks completed over time.

Description of Figure 1: This figure displays the rate at which each scheduler completes tasks with time. The steeper the curve, the faster tasks are being completed. The usefulness of this view is that the average duration by itself may conceal long-tail behavior. Tail behavior is important in cluster scheduling since just a handful of delayed tasks can block an entire workload.

4.2.4 | WebUI Operational Visuals

In live campaigns the WebUI served as operator-facing view to confirm that scheduler choices were consistent with observed system behavior. It is used to complement the numeric KPI tables displaying a real-time cluster state and workload progress.

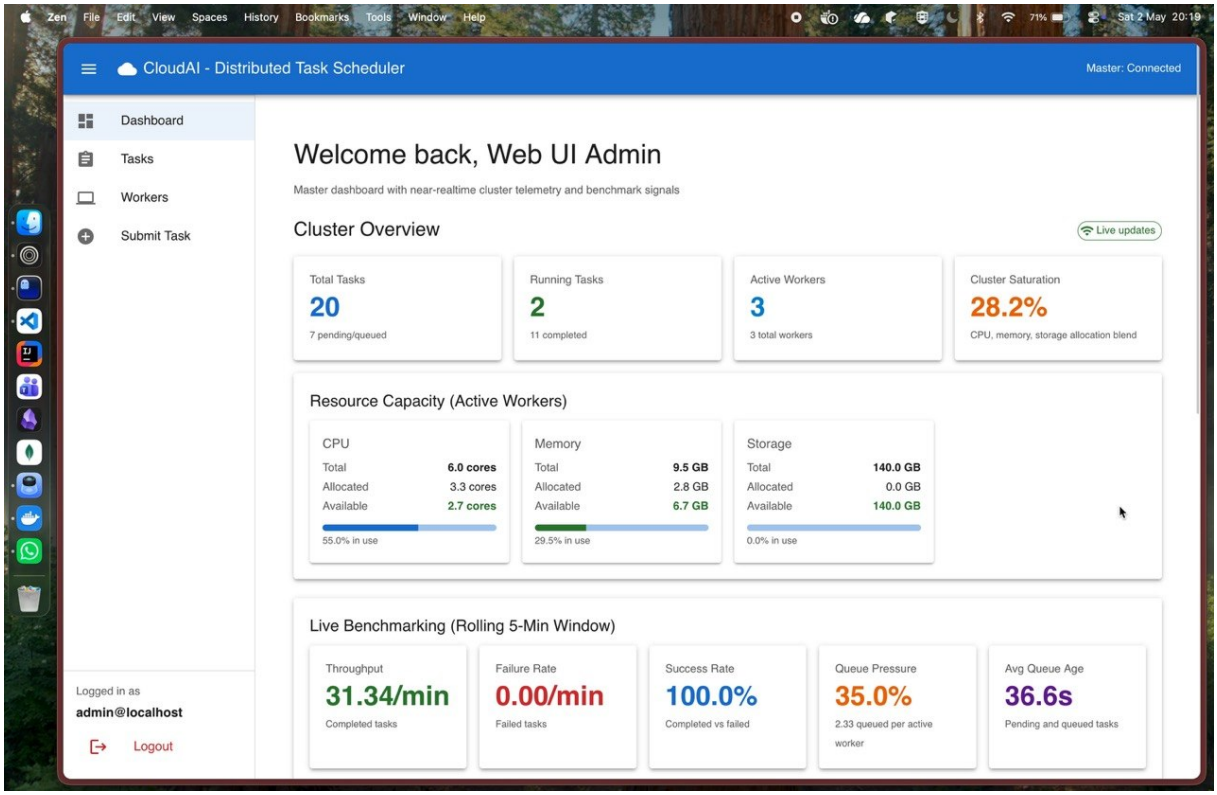

Figure 4.2.4: WebUI dashboard screen whilst running the campaign.

Description of Figure 4.2.4: The dashboard view gives an integrated status of the running scheduler, worker availability, and task lifecycle status. This served to confirm that the PPO placement decisions were not only better in the aggregate measures, but operationally consistent during the time that the workload was running.

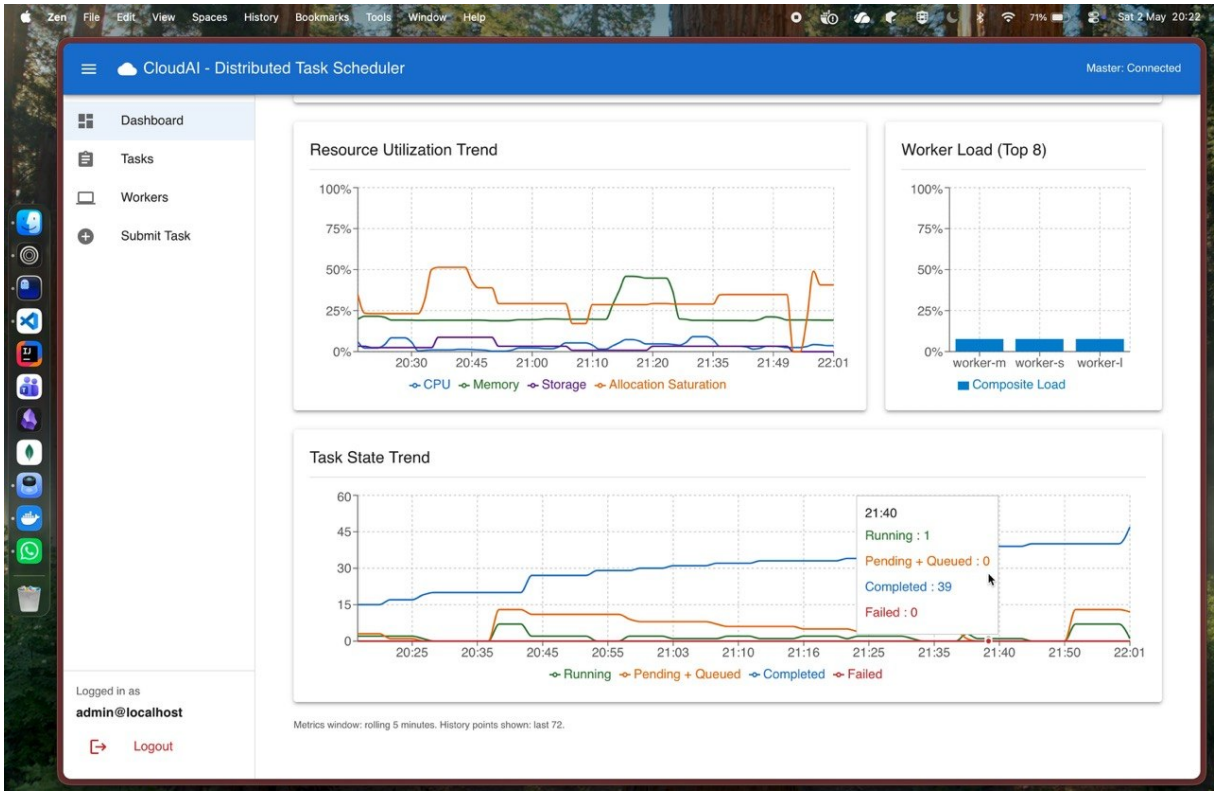

Figure 4.2.5: WebUI trend plot of scheduler performance monitoring.

Explanation of Figure 4.2.5: The trend plot visualizes runtime behavior across the campaign window, making it easier to inspect response under pressure and detect tail effects that may be hidden in simple averages. This visual evidence aligns with the measured improvements in turnaround and P95 completion for PPO in contention-heavy scenarios.

4.2.5 | Latest Campaign Summary

The campaign testbench is the end-to-end evaluation harness for the project. It runs against the live Go master over HTTP, switches the active scheduler, submits a fixed workload, polls task status until tasks become terminal or the timeout is reached, and then records duration, success rate, queue wait, average turnaround, and P95 turnaround. This is different from offline trace replay because the tasks are real Docker containers running on registered workers.

This campaign ran the `resource-contention-ppo` workload with 20 tasks per scheduler. All schedulers achieved 100% success, but PPO had the best overall runtime behavior. PPO finished the campaign in 63.95 seconds, compared with 70.06 seconds for Round-Robin and 76.20 seconds for RTS. PPO also had the lowest average turnaround and lowest P95 turnaround.

Table 4.2.2: New-model campaign scheduler summary

Scheduler	Tasks	Completed	Failed	Duration	Turnaround	P95
RR	20	20	0	70.06 s	33.80 s	69.00 s
RTS	20	20	0	76.20 s	33.40 s	74.00 s
PPO	20	20	0	**63.95 s**	**27.75 s**	**62.00 s**

Table 4.2.3: New PPO improvement over baselines

Comparison	Duration Improvement	Turnaround Improvement
PPO vs RR	8.7% faster	17.9% lower
PPO vs RTS	16.1% faster	16.9% lower

The difference came from execution placement: PPO selected workers in a way that reduced total campaign duration and reduced the tail turnaround. Turnaround time means the time from task creation to task completion. Tail turnaround focuses on the slowest part of the workload, usually measured using a high percentile such as P95. In this campaign, P95 turnaround means that almost all tasks completed within that many seconds, except the slowest few tasks. PPO reducing P95 from 69.00 seconds for Round-Robin and 74.00 seconds for RTS to 62.00 seconds means the slowest tasks were less delayed. This is important because a cluster can look acceptable on average while still having a few tasks that finish very late.

4.2.6 | Pressure Scenarios

The pressure tests were designed to make scheduling non-trivial. The testbench can run baseline, burst, and overload modes. In baseline mode, one copy of the workload is submitted and the runner waits up to 600 seconds for completion. In burst mode, the same workload is submitted without spacing between tasks, again with a 600 second per-scenario timeout. In overload mode, the workload is repeated three times to saturate the cluster, and the timeout is increased to 1200 seconds.

The `resource-contention-ppo` workload used in the final campaign contains 20 deterministic Docker tasks. The task set is mixed on purpose: five large CPU-heavy tasks, five memory-heavy tasks, five medium mixed CPU-memory tasks, and five small CPU-light tasks. The resource requests are also different. Large tasks request 2.5 CPU and 0.8 GB memory, memory-heavy tasks request 0.8 CPU and 2.0 GB memory, medium tasks request 1.5 CPU and 0.8 GB memory, and small tasks request 0.5 CPU and 0.4 GB memory. This creates a bin-packing problem because not every task is equally suitable for every worker.

Earlier comprehensive campaigns showed the clearest PPO advantage under burst load. In one campaign, Round-Robin and RTS timed out on multiple burst workloads, while PPO completed a large percentage of burst tasks. This supports the original motivation: learned scheduling matters most when the placement decision is non-obvious.

Table 4.2.4: Burst scenario interpretation

Observation	Interpretation
RR performs well under light load	Simple cyclic placement is enough when all workers have spare capacity.
RTS improves some resource-aware cases	Heuristics help when they match the workload pattern.
PPO improves under burst or overload pressure	Learned placement can capture interactions between task size, worker state, and tail risk.
PPO has overhead	A learned scheduler must justify its cost by improving difficult cases.

4.2.7 | Important Engineering Decisions and Results

Table 4.2.5: Key project decisions and outcomes

Decision	Reason	Outcome
Use Go for master and worker	The control plane needs efficient networking and concurrency.	The cluster can run master-worker task execution with clear service boundaries.
Use Docker for task execution	Tasks need portable execution environments.	Workers can run containerized workloads with resource requirements.
Use action masking in PPO	Invalid workers must not be selected.	PPO decisions respect hard resource constraints.
Use Lifecycle resource tracking during trace replay	Decay-based resource release was incorrect when simultaneous arrivals occurred.	PPO is controlled towards lessening hazardous placements.
Use fallback scheduler validation	A learned model can fail or be unavailable.	Cluster safety is maintained even when PPO cannot make a decision.

4.2.8 | Delivered Components

The final project is one that provides:

- a working distributed task execution cluster,
- worker registration and heartbeat tracking,
- task assignment and report of results,
- scheduler comparison infrastructure,
- PPO trace replay training,

- PPO service integration with Go,
- offline and end-to-end evaluation artifacts.

4.3 | My Contributions to the Project

The design and development of the Go cluster framework, the scheduler integration boundary, the PPO scheduling pipeline, the training and evaluation workflow, and the final benchmark interpretation were the main individual contributions. A significant portion of the work, as well, was debugging practical system issues: resource accounting, worker visibility, fallback behavior, and model loading.

4.4 | Learning Outcomes

The project served to bridge two topics which are frequently discussed independently: distributed systems and reinforcement learning. Part 6 of the distributed systems course has helped me understand that a lot of correctness is determined by the state handling, retries, and clean boundaries. The RL component taught that the quality of models is susceptible to reward design, feature representation, and safe deployment. The greatest lesson here was that the learning-based scheduling must be introduced as a controlled enhancement and not as a replacement of the basic system reliability.

4.5 | Real World Applications

This project is applicable to small cloud platforms, internal batch execution systems, university lab cluster, CI workload runners, and clusters of the edge where machines have varying capabilities. The PPO scheduler is particularly applicable when the workloads are mixed and the rules of manual scheduling cannot be easily tuned.

The structure can be employed in a university or research laboratory to share a small number of machines with many users who may be running experiments. Experiments might be CPU-intensive, might be memory-intensive, and might be short utility tasks. An educated scheduler can minimize the likelihood that a small job will be blocked behind an improperly placed large job.

The framework can be used in CI and testing systems to schedule build, test, and benchmark jobs on workers with varying capacities. A basic Round-Robin scheduler can allocate a heavy integration test to a slow machine and PPO can learn what it has done and prefer to place it somewhere better.

Machines in edge computing tend to be heterogeneous and resource-constrained. A scheduler should make decisions on the placement of tasks without necessarily having all the nodes with the same capacity. The master-worker concept can be applied to direct the processing of video, filtering of data, or inference tasks to appropriate edge nodes.

In the case of a private cloud, the project may serve as a lightweight batch execution

layer. It is not designed to compete with large orchestration platforms, but demonstrates how a focused scheduler can be constructed, tested, and trained with reinforcement learning and still maintain safe fallback behavior.

Chapter 5

Conclusion

The Agentic Cloud Cluster is a successful attempt to illustrate the distributed task execution framework based on Go with PPO-based scheduling. The practical necessity to distribute containerized jobs among heterogeneous workers grew into a full-fledged scheduling study. The last design isolates the cluster control plane and the learning service which ensures that the system is maintainable and safe.

The methodology demonstrated the reason why reinforcement learning is a viable choice to solve cluster scheduling. Q-tables describe the fundamental concept, but do not work due to the large size of the state space. DQN enhances generalization, and is less natural to dynamic sets of workers and masking of actions. PPO was selected due to its ability to directly learn a worker-selection policy, support stable clipped updates and continuous resource capabilities.

The outcomes demonstrate a balanced image. It is not always the case that PPO is better. Simple schedulers are quick and efficient when the load is light. PPO proves more handy under burst and overload pressure since it is capable of using acquired relationships between task demand, worker state and risk. This is the overall conclusion of the project: learning-based scheduling is useful when the decision space is complicated, but it needs to be implemented with validation and fallback.

5.1 | Future Work

Future work can improve the project in the following ways:

- Train on larger and more diverse traces.
- Add multi-objective reward tuning for cost, energy, and fairness.
- Improve online adaptation after longer stable cluster runs.
- Evaluate with more physical or cloud machines.
- Add stronger failure injection tests for worker loss and network delay.
- **Container Checkpoint/Restore with CRIU**: Integrate CRIU (Checkpoint/Restore In Userspace) to enable faster container recovery and task migration. CRIU provides

kernel-level support for freezing process state, dumping it to disk, and restoring it on demand. This capability would support several improvements:

- **Faster Recovery from Failures**: When a worker fails, instead of re-running tasks from scratch, containers can be restored from saved checkpoints, significantly reducing recovery time.
- **Task Preemption**: High-priority tasks could preempt lower-priority ones by checkpoint-saving the latter and restoring them when resources become available.
- **Container Migration**: Checkpoints enable live migration of containers between workers without loss of state, supporting better load balancing and maintenance operations.
- **Warm Starts**: Container images with pre-initialized state can be checkpointed and restored, reducing initialization overhead for stateful workloads.
- **Predictive Scheduling**: The scheduler could use CRIU-backed snapshots as part of risk modeling, allowing it to trade off immediate completion against future resource availability.

Implementation challenges include kernel compatibility, coordination between checkpoint timestamps and task state metadata in MongoDB, and handling network connections across migration boundaries.

Overall, the project proves that a distributed systems framework and a reinforcement learning scheduler can be combined in a practical and safe way.

Bibliography

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. MIT Press, 2 ed., 2018.

[2] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3-4, pp. 279–292, 1992.

[3] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing atari with deep reinforcement learning," *arXiv preprint arXiv:1312.5602*, 2013.

[4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[5] Alibaba, "Alibaba cluster data." https://github.com/alibaba/clusterdata, 2018. Cluster traces released for cluster management research.

[6] Kubernetes Authors, "What is kubernetes?." https://kubernetes.io/docs/concepts/overview/, 2026. Accessed: 2026-05-03.

[7] K. Ma, K. Wang, *et al.*, "Volcano: A cloud native batch system." https://github.com/volcano-sh/volcano, 2025.

[8] Apache Software Foundation, "Apache yunikorn: A cloud native scheduler." https://yunikorn.apache.org/, 2025.

[9] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan, and I. Stoica, "Ray: A distributed framework for emerging AI applications," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018.

[10] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," *International Conference on Machine Learning (ICML)*, 2018.

[11] Docker Inc., "Docker documentation." https://docs.docker.com/, 2026. Accessed May 2026.

[12] The Go Authors, "Documentation - the go programming language." https://go.dev/doc/, 2026. Accessed May 2026.

[13] gRPC Authors, "Introduction to grpc." https://grpc.io/docs/what-is-grpc/introduction/, 2026. Accessed May 2026.

[14] MongoDB, “Introduction to mongodb.” https://www.mongodb.com/docs/manual/introduction/, 2026. Accessed May 2026.

Appendix

.1 | Repository Modules

Table .1.1: Main repository modules used in the project

Path	Purpose
`master/`	Go master node, scheduler interface, task handling, worker handling, persistence hooks.
`worker/`	Go worker node, Docker task execution, heartbeat reporting, log streaming.
`proto/`	Protocol Buffer definitions for master-worker and PPO scheduler communication.
`agentic_scheduler/`	Python PPO model, service, training code, trace replay environment, checkpoints.
`testbench/`	Workload and campaign automation for end-to-end evaluation.
`results/`	Benchmark reports, campaign outputs, plots, and extracted result summaries.

.2 | Representative PPO Configuration

Table .2.1: Representative PPO training configuration

Parameter	Value / Meaning
Model	Actor-critic neural network with two hidden layers.
Hidden size	128.
Task feature dimension	5.
Worker feature dimension	9.
Clip ratio	0.2.
Value coefficient	0.5.
Entropy coefficient	Tuned during experiments, commonly around 0.01–0.02.
Training source	Alibaba trace replay and project campaign traces.

.3 | Ports and Service Mapping

Table .3.1: Ports used by the project

Component	Port / Address	Purpose
Master gRPC server	`0.0.0.0:50051`	Main worker-to-master communication endpoint for registration, task assignment coordination, heartbeats, and result reporting.
Master HTTP API	`0.0.0.0:8080`	HTTP API used for task, worker, file, auth, health, and metric endpoints.
PPO scheduler service	`127.0.0.1:50050`	Python gRPC service used by the Go PPO scheduler for `Ping`, model loading, worker selection, and outcome reporting.
Worker 1 gRPC	`0.0.0.0:50052`	Worker task execution endpoint.
Worker 2 gRPC	`0.0.0.0:50053`	Worker task execution endpoint.
Worker 3 gRPC	`0.0.0.0:50054`	Worker task execution endpoint.
Additional workers	`0.0.0.0:5005N`	Extra worker nodes can use the same pattern with unique ports.
MongoDB	`localhost:27017`	Persistent storage for tasks, workers, assignments, attempts, results, users, and scheduler model metadata.
Worker metrics	`0.0.0.0:9101+`	Per-worker metrics endpoint. Each worker can use a unique metrics port when multiple workers run on the same host.
Docker daemon	Host socket / Docker API	Container execution backend used by worker nodes. Exact access path depends on local Docker setup.

These mappings are the default local development mappings used by the project. In a multi-machine deployment, the same logical services remain, but the hostnames change from `localhost` to routable machine addresses.